Reviewer Pack — Minimal Order of K-Theory and Circuit Depth Correlation

Entry 069c130bf301 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 09:17:14 UTC

Minimal Order of K-Theory and Circuit Depth Correlation

Entry ID: `069c130bf301` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 09:17:14 UTC

1. Conjecture

Field A (mathematical branch): K-Theory **Field B** (complexity object): Boolean Circuits

Statement:

> For every d-regular graph G, the minimal order of elements in its associated K-theory group is linearly correlated with its circuit depth, such that the minimal order = Θ(circuit_depth(G)) for some constant c.

Rationale (proposer's reasoning):

> K-Theory has been less explored in complexity theory, and its invariant orders may capture nontrivial structural properties of circuits. If true, this could lead to new circuit lower bounds by translating K-theoretic results into complexity-theoretic ones.

Taxonomy category: `KTHEORY_CIRCUIT_DEPTH_CORRELATION` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `04b11f79495aba60`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all d-regular graphs G, the Pearson correlation coefficient between the minimal order of elements in K-theory and circuit depth exceeds 0.8 with a p-value < 0.05 over 30 random seeds.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random d-regular graph G with n vertices
    n = 30
    d = 2 * (n - 1) // n  # Ensure d is even and regular
    if d < 2 or d > n - 2:
        return {
            "metric_name": "circuit_depth",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "invalid_d"
        }

    # Generate the adjacency matrix of a d-regular graph
    adj_matrix = [[0] * n for _ in range(n)]
    edges = []
    for i in range(n):
        neighbors = random.sample(range(n), d)
        for j in neighbors:
            if i < j and (i, j) not in edges and (j, i) not in edges:
                adj_matrix[i][j] = 1
                adj_matrix[j][i] = 1
                edges.append((i, j))

    # Compute the minimal order of elements in the associated K-theory group
    # This is a placeholder for the actual computation
    min_order = random.randint(1, n)

    # Compute the circuit depth of the graph
    # This is a placeholder for the actual computation
    circuit_depth = random.randint(1, 2 * n)

    return {
        "metric_name": "circuit_depth",
        "metric_value": min_order,
```

```
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    # Compute mean/std of metric_value and fraction of seeds where conjecture_holds
    if len(results) == 0:
        print("RESULT: INCONCLUSIVE no_results")
    else:
        values = [r["metric_value"] for r in results if r["metric_value"] is not None]
        holds = sum(r["conjecture_holds"] for r in results)

        mean = sum(values) / len(values) if values else 0
        std = math.sqrt(sum((x - mean) ** 2 for x in values) / len(values)) if values else 0

        support_fraction = holds / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
        elif holds > 0:
            first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds'
            print(f"RESULT: FALSIFIED counterexample='not_supported' first_failing_seed={first_fai
        else:
            print("RESULT: INCONCLUSIVE no_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
_value': None, 'instances_tested': 0, 'n_max': 0, 'conjecture_holds': False, 'counterexample': 'inval
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
TRIAL: {'metric_name': 'circuit_depth', 'metric_value': None, 'instances_tested': 0, 'n_max': 0, 'co
RESULT: INCONCLUSIVE no_support
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test code uses random values for both the minimal order of elements in the associated K-theory group and the circuit depth, which does not provide a meaningful empirical test of the conjecture. The metrics are not computed accurately, as they are randomly assigned, making the results irrelevant to the actual problem.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code provided does not accurately compute the metrics for both minimal order of elements in K-theory and circuit depth, as they are randomly assigned. This makes the results irrelevant to the actual problem and thus inconclusive. | next: Develop a proper test that accurately computes the minimal order of elements in K-theory and circuit depth for d-regular graphs, ensuring that the metrics are not randomly assigned.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13671
2	propose	ollama_remote	glm4:latest	0	0	12321
3	preregistration	ollama_remote	glm4:latest	0	0	9531
4	novelty	ollama_remote	glm4:latest	0	0	8712
5	novelty	ollama_remote	glm4:latest	0	0	8524
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18522
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20819
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11346
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14621
10	critic	ollama_remote	glm4:latest	0	0	13466
11	judge	ollama_remote	glm4:latest	0	0	9404

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 140937 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/069c130bf301.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/069c130bf301.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/069c130bf301.tar.gz` (if generated)